Exame especial

A falha e a volta: DLQ e compensação no Mercado Rápido — 100 pontos

AED · Arquitetura Reativa e Event-Driven · Turma ASDO 11.1 · Prof. Sândalo Bessa · publicado em sexta-feira, 16/10/2026

1. O que é esta atividade

Esta é a atividade do exame especial da disciplina. Ela vale 100 pontos, é individual e tem sete dias corridos de prazo, como prevê o Plano de Ensino.

Ela cobra o mesmo núcleo técnico do projeto final — o caminho de falha e a compensação —, em escala de uma pessoa: sobre um código que você já tem, num domínio que você já conhece das aulas, sem documento de arquitetura e sem apresentação.

Em uma frase

Você vai ensinar o servico-estoque do Mercado Rápido a distinguir a falha que passa da falha que não passa, a mandar a segunda para uma DLQ, e a devolver uma reserva de estoque quando o pagamento do pedido for recusado — publicando isso como um fato novo.

1.1 — As datas

Quando	O quê	Onde
sexta-feira, 23/10/2026 23h59	O código, o README com o roteiro de teste, docs/adr/ADR-006-resiliencia.md, docs/IA.md e a tag exame-especial	GitHub, no seu repositório público
sexta-feira, 23/10/2026 23h59	O link do repositório, e nada além dele	Canvas, na tarefa do exame especial

1.2 — Individual, e o que isso quer dizer aqui

Conversar sobre conceito com colegas é permitido e saudável. O código, o ADR e o registro de IA são de cada um. Como não há prova nem apresentação, o histórico de commits é o que mostra o seu trabalho — por isso a seção 2 pede que o primeiro commit seja o código do ZIP sem alteração: tudo o que vem depois dele é seu, e fica visível.

2. Ponto de partida e repositório

2.1 — O código

Use o projeto demo-kafka-idempotencia do ZIP da aula 01 (os ZIP das aulas seguintes contêm o mesmo projeto, com mais serviços). Esta atividade usa só dois deles:

Serviço	Papel nesta atividade
servico-pedidos	publica PedidoConfirmado, como já faz. Pode ganhar um endpoint para publicar a recusa do pagamento (Parte B), se você escolher esse caminho
servico-estoque	é onde quase todo o trabalho acontece: retentativa, DLQ, a tabela das reservas e a compensação

O README do exemplo continua sendo o guia de como subir o ambiente: Docker Compose com Kafka na porta 19092 e Postgres na 15432, Java 21, Maven.

2.2 — O repositório

- Individual e público, criado na sua conta do GitHub — a mesma cujo nome de usuário é a sua matrícula;
- Nome: aed-2026-2-exame-<matrícula> — por exemplo, aed-2026-2-exame-250000;
- PRIMEIRO COMMIT: o conteúdo do ZIP, sem nenhuma alteração, com a mensagem "codigo de partida (ZIP da aula 01)";
- Do segundo commit em diante, o seu trabalho, em commits pequenos e com mensagem que diga o que mudou;
- Ao terminar, a tag exame-especial apontando para o commit que vale.

```
git tag -a exame-especial -m "Exame especial - AED"
git push origin exame-especial
```

Confira o e-mail do git antes do primeiro commit

git config user.email precisa ser um e-mail que o GitHub reconheça na sua conta. Se não for, o commit aparece como um nome solto, sem link para o seu usuário — e o histórico deixa de mostrar que o trabalho é seu.

2.3 — A árvore esperada

```
README.md                          o roteiro para reproduzir as Partes A e B
docs/
  adr/ADR-006-resiliencia.md       Parte C
  IA.md                            Parte D
docker-compose.yml
servico-pedidos/   ...
servico-estoque/   ...
```

Os caminhos dos documentos são literais: caminho diferente é caminho não encontrado.

3. Parte A — o caminho de falha (35 pontos)

Hoje, quando o servico-estoque não consegue processar um evento, a decisão do que fazer é tomada por omissão — pelo padrão do Spring Kafka. Esta parte a torna explícita.

3.1 — As duas falhas que você vai tratar

Tipo	No Mercado Rápido	O que fazer

Transitória	o Postgres está fora do ar (docker compose stop postgres)	retentar, com limite e com espera entre as tentativas
Permanente	o pedido traz um SKU que não existe na tabela estoque, ou quantidade ≤ 0	não retentar — vai direto para a DLQ, porque tentar de novo dá o mesmo erro

Um detalhe do código de partida que vale encontrar

O EstoqueRepository.reservar() faz um UPDATE por SKU. Para um SKU que não existe, o UPDATE afeta zero linhas, não lança erro nenhum — e o evento é marcado como processado. Hoje, a falha permanente é engolida em silêncio.

Transformar esse silêncio em uma falha com nome é o primeiro passo da Parte A.

3.2 — O que a Parte A exige

- Uma política de retentativa com LIMITE e espera entre as tentativas; o número é seu, e ele é justificado no ADR-006;
- A falha permanente é reconhecida no código — uma exceção própria, com nome do domínio — e NÃO é retentada;
- O tópico de DLQ: loja.pedido.confirmado.v1.dlq;
- O evento chega à DLQ com os cabeçalhos CloudEvents originais preservados — ce_id inclusive, porque é a chave de deduplicação de quem reprocessar;
- Junto com ele, o motivo: a exceção, a mensagem e o instante da falha;
- Um caminho de reprocessamento, mesmo que manual: corrigida a causa, o evento volta ao fluxo normal. Endpoint, consumidor ou script — a escolha é sua, e o README diz como usar;
- Reprocessar o mesmo evento duas vezes não debita o estoque duas vezes — a idempotência por ce_id que o projeto já tem é o que garante isso.

Dica de caminho, não obrigação: no Spring Kafka, DefaultErrorHandler com DeadLetterPublishingRecoverer e addNotRetryableExceptions resolve a maior parte disso por configuração. Se usar, o ADR explica o que ele faz por você — inclusive o que ele põe nos cabeçalhos.

4. Parte B — a compensação (35 pontos)

O caminho de exceção é o do case desde a aula 01: o pedido é confirmado, o estoque é reservado, e o pagamento é recusado. A reserva já aconteceu e outro consumidor pode já tê-la lido. Ela não se apaga: desfaz-se com um fato novo.

4.1 — Os dois eventos novos

Evento	Tópico	Quem publica	Quem consome
PagamentoRecusado	loja.pagamento.recusado.v1	quem simula o pagamento — um endpoint no servico-pedidos, ou um script/JSON publicado no tópico	servico-estoque
ReservaDeEstoqueCancelada	loja.estoque.reserva-cancelada.v1	servico-estoque, depois de devolver a quantidade	ninguém, por enquanto — e tudo

			bem: é o fato que fica registrado

Os dois com envelope CloudEvents em modo binário — os mesmos quatro atributos obrigatórios que o PedidoConfirmado já usa — e datas em ISO-8601.

4.2 — O que a Parte B exige

- O servico-estoque passa a registrar O QUE reservou para cada pedido (pedido, SKU, quantidade). Sem isso, ele não sabe o que devolver;
- Ao consumir PagamentoRecusado, ele devolve exatamente o que reservou para aquele pedido e publica ReservaDeEstoqueCancelada;
- A compensação é idempotente: a mesma recusa entregue duas vezes devolve o estoque uma vez só;
- O estado final é observável: a quantidade em estoque volta ao valor de antes, e o evento de compensação aparece no tópico (Kafka UI ou console);
- Nenhuma chamada HTTP entre os serviços para desfazer: o desfazer viaja como evento.

Compensação é um evento novo, nunca um DELETE no histórico

Apagar a linha da reserva, ou o evento do tópico, destrói a única coisa confiável do sistema: a sequência do que aconteceu. A reserva aconteceu; a devolução também aconteceu; as duas ficam registradas.

O nome segue a regra de sempre — particípio, linguagem do domínio, fato consumado. Nunca DesfazerReserva, que é comando, nem ReservaDeletada, que é vocabulário de banco de dados.

5. Parte C — o ADR-006 (20 pontos)

docs/adr/ADR-006-resiliencia.md, nas cinco seções do modelo que a disciplina usa desde a aula 02: Título, Status, Contexto, Decisão e Consequências aceitas. Ele justifica as decisões das Partes A e B:

- quantas tentativas, com que espera, e por quê;
- o que você classificou como transitório e como permanente, e por quê;
- como o evento sai da DLQ e volta ao fluxo, e quem faria isso em produção;
- por que a compensação é coreografada (o estoque reage a um fato) e não orquestrada — ou o contrário, se você escolheu diferente.

5.1 — As duas perguntas da arguição, por escrito

No lugar da apresentação, o ADR termina com uma seção "Perguntas" que responde, em poucos parágrafos, às duas perguntas da arguição de 14/09:

As duas perguntas

1. O que acontece se o servico-estoque cair NO MEIO da compensação — depois de devolver a quantidade e antes de publicar ReservaDeEstoqueCancelada?

2. Que consequência você aceitou ao escolher a sua política de retentativa?

A primeira não tem resposta única. O que se avalia é se a resposta olha para o código que você escreveu, e não para um diagrama genérico.

6. Parte D — o uso de IA (5 pontos)

docs/IA.md: o que a ferramenta sugeriu, o que você aceitou e o que você RECUSOU, com o motivo. Uma recusa técnica e específica vale mais que dez aceites.

Esta atividade é rica em recusa possível: peça à IA para "desfazer a reserva" e observe se ela sugere um DELETE; peça uma política de retentativa e observe se ela tem limite.

7. O README — como reproduzir

O README do repositório ganha uma seção no topo, "Exame especial — como reproduzir", com os comandos e a saída esperada para três cenários:

- 1. Falha permanente: publicar um pedido com SKU inexistente e ver o evento na DLQ, com o motivo;
- 2. Falha transitória: parar o Postgres, publicar um pedido, religar o Postgres e ver o evento processado depois das retentativas;
- 3. Compensação: publicar um pedido, conferir o estoque, publicar a recusa, conferir o estoque de volta e o evento ReservaDeEstoqueCancelada no tópico.

Testes automatizados no lugar de algum cenário são bem-vindos; nesse caso, o README diz qual teste cobre qual cenário e como rodá-lo.

8. Checklist antes de criar a tag

- 1. O repositório é público e se chama aed-2026-2-exame-<matrícula>.
- 2. O primeiro commit é o ZIP sem alteração; os seguintes estão ligados à sua conta.
- 3. A falha permanente tem exceção própria, com nome do domínio, e não é retentada.
- 4. A retentativa tem limite e espera.
- 5. O evento chega a loja.pedido.confirmado.v1.dlq com ce_id e com o motivo.
- 6. Existe um jeito de reprocessar o que está na DLQ, e ele está no README.
- 7. Reprocessar duas vezes não debita duas vezes.
- 8. O estoque registra o que reservou por pedido.
- 9. PagamentoRecusado devolve a quantidade e publica ReservaDeEstoqueCancelada.
- 10. A mesma recusa entregue duas vezes devolve uma vez só.
- 11. O ADR-006 tem as cinco seções, a seção Perguntas, e ao menos uma consequência negativa nomeada.
- 12. docs/IA.md tem pelo menos uma recusa com motivo.
- 13. A tag exame-especial existe e foi enviada com git push origin.

- 14. O link do repositório foi enviado no Canvas.

9. Rubrica

Publicada junto com o enunciado, como em todas as etapas. A devolutiva é o nível marcado por critério, mais um comentário curto. Os pesos são pontos, de um total de 100.

Critério	Pontos	Insuficiente	Adequado	Exemplar
O caminho de falha (Parte A)	35	retentativa sem limite, ou falha permanente ainda engolida; DLQ ausente ou sem o motivo	falha permanente com exceção própria e não retentada; DLQ com ce_id e motivo; reprocessamento no README	o reprocessamento é demonstrado e comprovadamente idempotente; os dois cenários são reproduzíveis por teste
A compensação (Parte B)	35	o desfazer é DELETE, UPDATE sem evento, ou chamada HTTP entre os serviços	devolve o que foi reservado, publica ReservaDeEstoqueCancelada com CloudEvents, estado final observável	idempotente, e trata a recusa de um pedido sem reserva (que chegou antes, ou nunca reservou)
ADR-006 e as duas perguntas	20	seções faltando, consequências só com vantagens, ou seção Perguntas ausente	as cinco seções, alternativa descartada, consequência negativa nomeada, as duas perguntas respondidas	a resposta à pergunta 1 aponta o ponto exato do código e o que o sistema faz para se recuperar
Uso crítico de IA	5	docs/IA.md ausente, ou sem nenhuma recusa	sugestões, aceites e ao menos uma recusa com motivo	a recusa é técnica e específica, e mostra critério que a ferramenta não tinha
Repositório, histórico e prazo	5	caminhos fora do padrão, tag ausente, ou commits sem autoria ligada à conta	caminhos corretos, primeiro commit intacto, tag publicada no prazo	o histórico conta a ordem em que o trabalho foi feito, em commits pequenos e com mensagens claras

10. O que esta atividade não pede

Não se pede documento de arquitetura, apresentação, vídeo, serviço de pagamento de verdade, nem interface de operação para a DLQ. Também não se pede o domínio da sua equipe: o recorte é o do Mercado Rápido, para que todos partam do mesmo código.

Dúvida de escopo vai pela caixa de mensagens do Canvas, endereçada ao professor; a resposta que valer para todos é repassada aos demais.

Prazo

Até 23/10, 23h59. Os sete dias corridos são o que o regulamento prevê para o exame especial, e a correção começa logo depois — vale enviar o link no Canvas com alguma folga.